Hands on Generative AI: Latent Diffusion Model

Giacomo Negri & Marco Zimmatore

Abstract

This report aim to provide a comprehensive explanation of Score-based Diffusion Models and information relating to our technical implementation of a Latent Score-based Diffusion Model (LDM) inspired by Song et al.(1) and Rombach et al.(2). Our objective is to develop from scratch an LDM and deploying it effectively with our limited computational resources.

1. Introduction

Generative modeling aims to learn the underlying distribution $p_{\text{data}}(\mathbf{x})$ of a dataset $\{\mathbf{x}_i\}_{i=1}^N$ to synthesize new samples. Traditional approaches fall into two broad categories:

1. **Likelihood-based models:** directly approximate the density via Maximum Likelihood Estimation (MLE) (e.g., VAEs, EBM, Normalizing Flows). These often require architectural constraints (invertibility) or rely on surrogate objectives (ELBO).

2. **Implicit models:** learn to sample without explicitly modeling the density (e.g., GANs). These are known for training instability and mode collapse.

Diffusion models, or score-based generative models, bridge these gaps. They corrupt data with noise and learn to reverse this corruption. Specifically, by estimating the *score function* $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$, we can traverse the manifold of probability distributions from pure noise back to the data distribution.

Our implementation operates in a latent space, where a pretrained VAE (3) encodes images $\mathbf{x} \in \mathbf{R}^D$ into latents $\mathbf{x}_0 \in \mathbf{R}^d$, where $d < D$. The diffusion process is then applied to encoded images, reducing computational cost while maintaining perceptual quality. Specifically, our implementation is composed by five stages:

1. **Encoding**: the image compression is achieved through `stabilityai/sd-vae-ft-mse` (2).

2. **Forward passage**: executed through the closed form implementation of a continuous stochastic differential equation (SDE).

3. **Backbone training**: the U-Net model (1) was readapted for our latent representation and trained for conditional generation.

4. **Denoising**: the noised images are feeded recursevely into the network and their values updated while denoising them and retrieving the original data distribution.

5. **Decoding**: denoised images are feeded into the decoder of the VAEs to be decompressed and return an image of the original size.

These entire pipeline is well summarize by the following representation:

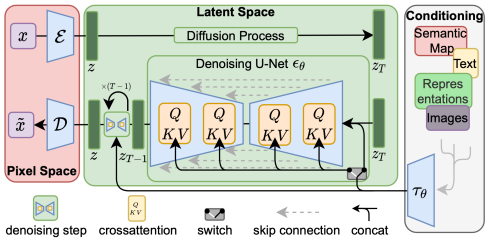

Figure 1. The architecture of the latent diffusion model (LDM). (Source: High-Resolution Image Synthesis with Latent Diffusion Models (2))

The actual implementation code can be found at the following GitHub repository in the references (4).

2. Variational Autoencoders (VAEs)

VAEs are an evolution of Autoencoders (AEs) that no longer limit themselves to learn a deterministic mapping, but instead they apprehend to map the input to a distribution, p_θ, where $\mathbf{x} \in \mathcal{D}$ and $\mathcal{D}$.

We define $g_\phi(\cdot)$ as the encoding function and $f_\theta(\cdot)$ as the decoding one, where $q_\phi(\mathbf{z}|\mathbf{x})$ is the estimated posterior of the probabilistic encoder and $p_\theta(\mathbf{x}|\mathbf{z})$ is the probabilistic decoder, or the likelihood of generating a true data samples.

Moreover, $\mathbf{x}'$ is the reconstructed version of $\mathbf{x}$ and $\tilde{\mathbf{x}}$ is the corrupted version of $\mathbf{x}$, while $\mathbf{z}$ is the compressed code learned in the bottleneck layers, so that $\mathbf{z} = g_\phi(\mathbf{x})$ and $\mathbf{x}' = f_\theta\left(g_\phi(\mathbf{x})\right)$, as done in (3).

Because when training an AE we are learning an identity function then the risk of overfitting is high, therefore as to reduce it, we corrupt the input by adding noise or masking same values: $\tilde{\mathbf{x}} \sim \mathcal{M}_{\mathcal{D}}(\tilde{\mathbf{x}}|\mathbf{x})$.

Since we are mapping to a distribution with VAEs, we have prior $p_\theta(\mathbf{z})$, likelihood $p_\theta(\mathbf{x}|\mathbf{z})$, and a posterior $p_\theta(\mathbf{z}|\mathbf{x})$. Neverthless, since solving for every image is computationally expensive, we approximate using the function $q_\phi(\mathbf{z}|\mathbf{x})$, so that now we have a probabilistic decoder $p_\theta(\mathbf{x}|\mathbf{z})$ and encoder $q_\phi(\mathbf{x}|\mathbf{z})$.

Since the estimated posterior should be very close to the actual one, we can minimize the Kullback Leiber distance, which mean maximising the likelihood of generating real data while minimizing the difference between the real and estimated posterior:

$$\begin{aligned} L_{\text{VAE}}(\theta, \phi) = -\mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})} \log p_\theta(\mathbf{x}|\mathbf{z}) \\ + D_{KL}(q_\phi\left(\mathbf{z}|\mathbf{x}\right) \| p_\theta(\mathbf{z})) \end{aligned} \tag{1}$$

3. Score-based Model

Score based model derive from Energy-based Model (EBM), where the probability density function (pdf) is computed as: $f_\theta(x) : \mathbb{R}^n \to \mathbb{R}$, parametrized by θ:

$$p_\theta(\mathbf{x}) = \frac{e^{-f_\theta(\mathbf{x})}}{z_\theta}$$

where $z_\theta > 0$ is a normalizing constant s.t. $\int p_\theta(\mathbf{x})\mathrm{d}\mathbf{x} = 1$. $f_\theta(x)$. Score function avoid the need to deal with the intractable denominator by taking the derivative of the logarithmic of the pdf:

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}) = -\nabla_{\mathbf{x}} f_\theta(\mathbf{x})$$

By consequence our objective becomes to minimize the Fisher Divergence, between the ground truth, $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ and the score computed by our model, $s_\theta(\mathbf{x})$. This can be achieved through Score-Matching, even without knowing the ground truth:

$$J(\theta) = \mathbb{E}_{p(\mathbf{x})}\left[\frac{1}{2}\|s_\theta(\mathbf{x})\|^2 + \nabla \cdot s_\theta(\mathbf{x})\right] \tag{2}$$

where $\nabla\cdot$ is the divergence operator. In this case $\nabla \cdot s_\theta(\mathbf{x}) = \mathrm{Tr}(\nabla_{\mathbf{x}} s_\theta(\mathbf{x}))$, the trace of the Jacobian.

Computing the Jacobian is expensive. Fortunately, Denoising Score Matching (DSM) (5) provides a more efficient equivalent using conditional score. Instead of matching the score of the complex and unknown $p_{data}(\mathbf{x})$, we perturb the data with known noise distribution $q_\sigma(\tilde{\mathbf{x}}|\mathbf{x})$ and match the score of the perturbated distribution. The objective becomes:

$$\begin{aligned} \mathcal{L}_{DSM}(\theta) = \mathbb{E}_{p_{data}(\mathbf{x})}\Big[\\ \mathbb{E}_{q_\sigma(\tilde{\mathbf{x}}|\mathbf{x})}\Big[\frac{1}{2}\|s_\theta(\tilde{\mathbf{x}}) - \nabla_{\tilde{\mathbf{x}}} \log q_\sigma(\tilde{\mathbf{x}}|\mathbf{x})\|_2^2\Big]\Big] \end{aligned} \tag{3}$$

Since we choose the noise distribution, typically a Gaussian, the term $\nabla_{\tilde{\mathbf{x}}} \log q_\sigma(\tilde{\mathbf{x}}|\mathbf{x})$ is analytically tractable. Intuitively, the model learns to point from a noisy sample $\tilde{\mathbf{x}}$ back toward the clean data $\mathbf{x}$, effectively learning the vector field that denoises the space.

In our implementation, we define the perturbation kernel as a Gaussian marginal $p_{0t}(\mathbf{x_t}|\mathbf{x}_0) = \mathcal{N}\left(\mathbf{x}_t; \mu(t)\mathbf{x}_0, \sigma^2(t)\mathbf{I}\right)$ (6). Under this specific noise schedule, the ground truth conditional score becomes:

$$\nabla_{\mathbf{x}_t} \log p_{0t}(\mathbf{x_t}|\mathbf{x_0}) = -\frac{\mathbf{x_t} - \mu(t)\mathbf{x}_0}{\sigma^2(t)}$$

and through the reparameterization trick, we express the noisy sample as $\mathbf{x}_t = \mu(t)\mathbf{x}_0 + \sigma(t)\epsilon$, where $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, therefore:

$$\nabla_{\mathbf{x}_t} \log p_{0t}(\mathbf{x_t}|\mathbf{x_0}) = -\frac{\epsilon}{\sigma(t)} \to s_\theta(\mathbf{x}_t, t) \approx -\frac{\epsilon_\theta(\mathbf{x}_t, t)}{\sigma(t)}$$

so that we can approximate the score. To stabilize and to speed up the learning, we have defined the model $s_\theta(x_t, t)$ to approximate the scaled score. Replacing this expression in the objective function of DSM, we obtain:

$$\mathcal{L}_{simple} = \mathbb{E}_{x_0,\epsilon,t}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|_2^2\right]$$

4. Stochastic Differential Equations (SDEs)

4.1. Forward process

We model the diffusion process as a continuous-time SDE. Let $\mathbf{x}_t \in \mathbb{R}^d$ be the latent variable at time $t \in [0, T]$, where for $t = 0$ it represents the data distribution and for $t = T$ it represents a known prior, in our case $\pi(\mathbf{x}) \sim \mathcal{N}(0, I)$.

The forward Itô SDE is given by:

$$\mathrm{d}\mathbf{x}_t = \mathbf{f}(\mathbf{x}_t, t)\mathrm{d}t + g(t)\mathrm{d}\mathrm{w}_t \tag{4}$$

where $f(\cdot, t) : \mathbb{R}^d \to \mathbb{R}^d$ is the drift, $g(t) \in \mathbb{R}$ is the diffusion, and w_t is a brownian motion, where dw as infinitesimal white noise. The solution of the SDE is a collection of random variables $\{X(t)\}_{t\in[0,T]}$, where $p_t(\mathbf{x})$ is the marginal pdf of $\mathbf{x}$ at time t.

4.2. Reverse process

In order to be able to generate data, we must reverse the diffusion process, which is given by another SDE (7):

$$\mathrm{d}\mathbf{x}_t = \left[\mathbf{f}(\mathbf{x}_t, t) - g^2(t)\nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t)\right] \mathrm{d}t + g(t)\mathrm{d}\bar{\mathbf{w}}_t \quad (5)$$

where dt represents a negative time step ($t : T \to 0$) and $\bar{\mathbf{w}}_t$ is Brownian motion in reverse time.

The only unknown term in the 12 is the score function $\nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t)$. We approximate this with a neural network $s_\theta(\mathbf{x}_t, t)$, which in our case is a U-Net.

4.3. Specific SDE

Given different types of SDE: Variance Preserving (VP), Variance Exploding (VE) and subVP, as presented in (8), our choice fell on the latter. The motivation for choosing it was that combines the best characteristics of two other SDE processes. A higher variance, VE, enable the model to retain more expressivity, while a stable variance allows to converge more easily. By applying a discount factor to the variance, subVP SDE tries to achieve both. Follow the specific formulations, such as for VE:

$$\mathrm{d}\mathbf{x} = \sqrt{\frac{\mathrm{d}[\sigma(t)^2]}{\mathrm{d}t}}\mathrm{dw}$$

VP formulation:

$$\mathrm{d}\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x}\mathrm{d}t + \sqrt{\beta(t)}\mathrm{dw}$$

and subVP formulation:

$$\mathrm{d}\mathbf{x} = \underbrace{-\frac{1}{2}\beta(t)\mathbf{x}}_{\text{drift: } \mathrm{f}(\mathbf{x}_t, t)}\,\mathrm{d}t + \underbrace{\sqrt{\beta(t)\left(1 - e^{-2\int_0^t \beta(s)\,\mathrm{d}s}\right)}}_{\text{diffusion: } g(t)}\,\mathrm{d}\mathbf{w}$$

5. Backbone

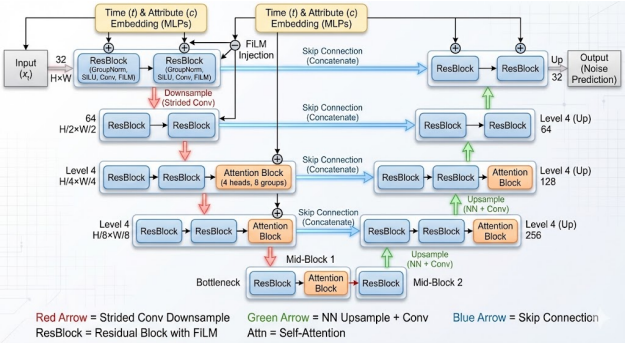

Figure 2. Architectural Overview of the Conditional U-Net.

We decided to implement a U-Net with a symmetric encoder-decoder structure with the following key components:

- **Encoder** (Downsampling): the input $\mathbf{x}_t$ is processed through a four-stage hierarchy. At each level, the data passes through a sequence of 2 ResBlock followed by downsampling convolutions. This structure progressively expands the feature dimensions from 32 to 256 channels while compressing spatial resolution. To balance computational efficiency with global context modeling, Self-Attention mechanisms are selectively activated only at the two deepest levels.
- **Bottleneck**: a central mid-block processes the most compressed representation (256 channels), where the receptive field is largest, facilitating the learning of high-level semantic features. This stage consists of two additional Residual Blocks, the first of which includes a global attention layer.
- **Decoder** (Upsampling): the compressed features are mapped back to the original input resolution through nearest-neighbor upsampling and subsequent residual blocks.
- **Skip Connections**: To mitigate the loss of spatial information during downsampling, feature maps from the encoder are concatenated with those in the decoder at corresponding resolutions. This allows the model to recover fine details necessary for accurate noise estimation.

Upsampling and Downsampling In the encoder, instead of using fixed operations like *Max-Pooling* or *Average-Pooling*, we utilize *Strided Convolutions* with kernel size 3 and stride $= 2$.
Pooling strategies are commonly used for classification tasks, because it is optimal for understanding if there is a feature but it is suboptimal for generative tasks, since it discards useful information. A strided convolution, instead, learns a weighted combination of all the pixels of the window, maintaining the most useful informations for the denoising.
Standard upsampling layers operate by projecting input pixels onto a larger grid. Mathematically, if the kernel size and stride do not match perfectly, this projection creates uneven overlaps, resulting in a grid-like noisy pattern. For a diffusion model, whose primary goal is denoising, generating structural noise is highly counterproductive.
So we use *Upsample* with *mode='nearest'*, a deterministic operation that expands the spatial dimensions by copying, ensuring that there are absolutely no overlapping artifacts.

SiLU The activation function used is **SiLU** ($x \cdot \sigma(x)$). Unlike ReLU, SiLU is continuously differentiable. This smoothness aids the optimization landscape, which is particularly complex in generative modeling where the manifold is high-dimensional.

GroupNorm We utilize **Group Normalization** (with 8 groups) instead of Batch Normalization. In diffusion models, memory constraints often require small batch sizes. Batch Normalization statistics become unstable and noisy under these conditions. Group Normalization is independent of batch size, providing stable feature standardization for single-sample generation.

Residual Block The core computational unit of our architecture is the **ResBlock**. Each block follows a pre-activation structure: `GroupNorm` $\rightarrow$ `SiLU` $\rightarrow$ `Conv` $\rightarrow$ `FiLM` $\rightarrow$ `Conv`.

Attention Mechanism To capture long-range dependencies that convolutions might miss, we integrate **Self-Attention** blocks. However, the computational cost of attention scales quadratically with the spatial resolution ($O(N^2)$). To balance expressiveness with efficiency, we apply attention *only at the lower resolutions*.
Our implementation utilizes a robust **Pre-Norm** [1] architecture inspired by DDPM++ and NCSN++: unlike original Transformer designs, we apply Group Normalization (with 8 groups) immediately *before* the attention operation, a configuration that significantly enhances gradient flow and training stability in deep generative networks. For the projection of Queries, Keys, and Values, the model employs 1×1 convolutions (Network-in-Network) rather than standard dense linear layers. This specific design choice treats spatial positions as individual tokens while preserving the underlying 2D grid structure of the feature maps. The final aggregated features are projected and added back to the input via a residual connection, ensuring the module refines the local features with global context rather than replacing them.

Time and Attribute Conditioning Since the score function $\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t)$ is time-dependent, the neural network must be conditioned on the diffusion time t. We utilize Sinusoidal Position Embeddings to map the continuous scalar $t \in [0, T]$ into a high-dimensional vector space.
Given a dimension d, the embedding for time t is calculated as:

$$PE_{(t,2i)} = \sin(t \cdot \omega_i) \tag{6}$$

$$PE_{(t,2i+1)} = \cos(t \cdot \omega_i) \tag{7}$$

where frequencies ω_i follow a geometric progression from 1 to 10000.
Unlike learned embeddings which are discrete, sinusoidal functions allow the model to generalize to arbitrary time steps, essential for continuous-time diffusion (SDEs). In addition, Neural networks suffer from "*spectral bias*" (learning low frequencies first). High-frequency sinusoidal features enable the network to distinguish between fine-grained noise levels (small t) and large structural corruption (large t). This embedding is further processed by a Multi-Layer Perceptron (MLP) and injected into every residual block via FiLM.

Feature-wise Linear Modulation (FiLM) The FiLM operation scales and shifts the normalized intermediate feature maps $\mathbf{h}$ based on the time embedding $\mathbf{e}_t$:

$$\mathbf{h}_{\text{cond}} = \mathbf{h} \odot (1 + \gamma(\mathbf{e}_t)) + \beta(\mathbf{e}_t) \tag{8}$$

where γ and β are affine transformations of the embedding, and $\odot$ denotes element-wise multiplication. Similarly, class labels or attributes are encoded via an attribute MLP and integrated with the time embedding to provide conditional guidance during the reverse process. This mechanism allows the time step t and attributes c to globally modulate the feature maps. By scaling and shifting the normalized features, the model effectively switches its operating mode, adapting the denoising behavior to the current noise levels (the model will be focusing on shapes at high noise vs. textures at low noise).

5.1. Functional Role in Score Matching

Within the context of denoising score matching, the network is trained to predict the scaled score function $\sigma_t \nabla_x \log p_t(x)$, which is equivalent to predicting the noise ϵ added to the data. The U-Net architecture is particularly suited for this objective because the score field shares the same dimensionality as the input $\mathbf{x}$. Furthermore, the residual connections enable the network to focus on learning high-frequency components (the noise) while bypassing the low-frequency structural information preserved through the skip paths.

6. Classifier and Classifier-Free Guidance

After having trained unconditionally we explored conditional generation through Classifier-Free Guidance (CFG) (9). CFG takes inspiration from Classifier Guidance (10), in which an auxiliary time-dependent classifier $p_t(c|\mathbf{x}_t)$ is trained on noisy data to predict the conditioning signal c. During the reverse process, the gradients of this classifier are used to "steer" the score-based model toward specific region of the probabilistic space. While effective, this approach is computationally expensive as it requires training and maintaining an additional model.

To address these limitations, CFG was introduced. During training, the conditioning signal c, often embedded classes or text, is randomly replaced by a "null" token $\emptyset$ with a fixed probability p_{uncond} (typically 10%–20%). This enables a single network to estimate both the conditional and unconditional scores. At inference, the guided score is extrapolated

[1] Pre-Norm design applies normalization before the main transformation within each residual block

as a linear combination:

$$\mathbf{s}_{\text{cfg}}(\mathbf{x}_t, t, c) = \mathbf{s}_\theta(\mathbf{x}_t, t, \emptyset) + \omega \cdot \big(\mathbf{s}_\theta(\mathbf{x}_t, t, c) - \mathbf{s}_\theta(\mathbf{x}_t, t, \emptyset)\big) \tag{9}$$

where ω is the guidance weight. This formulation isolates the implicit classifier gradient without requiring an external model. The intuition is that the vector difference between the conditional and unconditional scores points directly toward the desired class manifold.

7. Likelihood Weighting and Importance Sampling

In standard score-base diffusion models, we train through a weighted combination of score matching losses:

$$\mathcal{J}_{SM}(\theta, \lambda(\cdot)) = \frac{1}{2}\int_0^T \mathbb{E}_{p_t(\mathbf{x})}\Big[\lambda(t)\left\|\nabla_{\mathbf{x}} \log p_t(\mathbf{x}) - s_\theta(\mathbf{x}, t)\right\|_2^2\Big]\, \mathrm{d}t \tag{10}$$

where $\lambda(t)$ is a positive weighting function. (8) demonstrated that specific weighting can bound the negative log-likelihood of tehe data enabling a maximum likelihood estimation.

Likelihood Weighting (LW) Choosing $\lambda(t) = g(t)$ from Eq. 4, the objective $\mathcal{J}_{SM}$ becomes an upper bound on the KL divergence between p_{data} and the model distribution p_θ:

$$D_{KL}(p_{data} \| p_\theta) \leq \mathcal{J}_{SM}\left(\theta, g(\cdot)^2\right) + D_{KL}(p_T \| \pi)$$

this promotes higher likelihoods, but not necessarily sample quality, which seem to be optimized heuristically by $\lambda(t) \propto 1/\mathbb{E}\left[\|\nabla_{\mathbf{x}} \log p_{0t}(\mathbf{x}|\mathbf{x_0})\|^2\right]$.

Importance Sampling (IS) Using LW often increases variance, as the loss can vary significantly with t. To address this IS isntead of sampling t uniformly $[0, T]$, it is sampled from a proposal distribution $p(t) \propto g(t)^2/\alpha(t)^2$, where $\alpha(t)^2$ is a variance-reducing weighting function, usually the previously discussed heuristic weight.

8. Sampling Methods

Since the reverse diffusion process does not admit a closed-form solution, it must be discretized. In this implementation, we employ the Euler-Maruyama approximation to solve the underlying SDE. The transition step is defined as:

$$\mathbf{x}_{t-\Delta t} = \mathbf{x}_t + \mathbf{f}(\mathbf{x}_t, t)\Delta t + \mathbf{g}(t)\sqrt{\Delta t}\mathbf{z} \tag{11}$$

where $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$. In the context of score-based modeling, this step uses the learned score function to iteratively move from noiser distribution back to the data manifold.

9. Evaluation Metrics

In order to assess image generative models there are two types of metrics: likelihood-based metrics and perceptual quality metrics.

9.1. Negative Log-Likelihood (NLL)

Negative Log-Likelihood estimation is central to generative models since they are trained to maximixe the likelihood of the data. In diffusion processes the log-likelihood $\log p_\theta(\mathbf{x})$ is intractable due to the integration over latent variables. Nevertheless, it has been shown (8) that the ODE has the same marginal probability density $\{p_t(\mathbf{x})\}_{t=0}^T$ as the SDE used during training, but it follows a deterministic trajectory, therefore it could be expressed as:

$$\mathrm{d}\mathbf{x}_t = \left[\mathbf{f}(\mathbf{x}_t, t) - g^2(t)\nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t)\right]\mathrm{d}t \tag{12}$$

Through the change of variables formula we can compute the exact change in log-density via ODE. Let's define $\tilde{\mathbf{f}}_\theta$ as the drift of Eq. 12, then we can compute the log-likelihood of a data point $\mathbf{z}(0)$, in our case a latent representation, as follows:

$$\log p_0\left(\mathbf{z}(0)\right) = \log p_T\left(\mathbf{z}(T)\right) + \int_0^T \nabla \cdot \tilde{\mathbf{f}}_\theta\left(\mathbf{z}(t), t\right)\mathrm{d}t \tag{13}$$

where p_T is the known prior distribution and ∇ is the divergence operator.

Calculating the $\nabla \cdot \tilde{\mathbf{f}}_\theta$ is computationally expensive as discussed in Eq. 2, but it can be approximated by the Hutchinson Trace estimator. For a gaussian random noise $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, the approximation is:

$$\nabla \cdot \tilde{\mathbf{f}}_\theta = \mathbb{E}_\epsilon\left[\epsilon^T \nabla_{\mathbf{z}} \tilde{\mathbf{f}}_\theta(\mathbf{z}, t)\epsilon\right]$$

so we compute the vector-Jacobian product without computing the full Jacobian Matrix, and we integrate the ODE from $t = \epsilon$ to T using an Euler discretization scheme.

Must be noted that because our ODE is computed in latent space, and not in pixel space, we will observe lower values than in standard pixel-space benchmarks. This is due to the Evidence Lower Bound (ELBO) that influence the training of the VAE's encoder. The NLL of an image $\mathbf{x}$ is bounded by the objective (3):

$$-\log p(x) \leq \underbrace{-\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\psi(\mathbf{x}|\mathbf{z})]}_{\text{Reconstruction term}} + \underbrace{D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})\|p(\mathbf{z}))}_{\text{Prior matching}} \tag{14}$$

where the *Reconstruction term* measures the information lost during the compression $\mathbf{x} \to \mathbf{z}$, so it accounts for pixel-wise error. This usually dominates in pixel-space models,

because of high-frequency details and noise. The second, *Matching prior* measures the cost of encoding the latent variable $\mathbf{z}$ under the prior distribution. In our LDM, the diffusion model $p_\theta(\mathbf{z})$ acts as a learned, expressive prior.

In our ODE the $-\log p(\mathbf{x})$ corresponds strictly to the *Matching prior*, since we outsourced the expensive modeling of high-frequency details to the VAE's *Reconstruction term*. By consequence our NLL reflects only the *semantic compression* (2).

9.2. Fréchet Inception Distance (FID)

The FID score measures the similarity between the distribution of generated images and real images inthe feature space of a pre-trained Inception-v3 network. Differently from pixel-level metrics it correlates well with human perceptual judgement by approximating the features representation as a multidimensional Gaussian distribution.

The distance is calculated as a Wasserstein-2 distance between the Gaussians:

$$\text{FID} = \|\mu_\tau - \mu_g\|^2 + \mathbf{Tr}\left(\Sigma_r + \Sigma_g - 2(\sigma_r\Sigma_g)^{1/2}\right)$$

where (μ_r, Σ_r) and (μ_g, Σ_g) are the mean and covariance of the real and generated feature distributions, respectively.

In our implementation we used the `torchmetrics` library for a standardized implementation, were we first compute statistics for real images from the test set and then we generate and equal number of samples. Crucially, since our model works in the latent space of a VAE, the generated latents are decoded into pixel space and clamped before feature extraction, ensuring a fair final output comparison.

9.3. Inception Score (IS)

The inception score analyse the generated images based on two criteria: clarity and diversity, so that th image should look like a specific object/category and it should also be inclusive of a wide range of objects. IS works by computing the KL divergence between the conditional class distribution $p(y|x)$ and the marginal clas distribution $p(y)$:

$$\text{IS} = \exp\left(\mathbb{E}_{\mathbf{x}\sim p_g}\left[D_{KL}\left(p(y|\mathbf{x})||p(y)\right)\right]\right)$$

In this case we try to maximise the divergence because we would like $p(y|\mathbf{x})$ to have low entropy, hence clarity, while $p(y)$ to have high entropy, hence diverse classes across the generated set.

Similarly to before we use `torchmetrics` to compute IS on the same batch of decoded, generated images. In our case, while IS is less robust than FID, given that our dataset is not an ImageNet one, we report it to facilitate comparison across different SDE methods and architectures.

10. Experiments

10.1. Experimental setup

We explored our LDM on the CelebA dataset (11), using the pretrained VAE by Stability AI (`stability/sd-vae-ft-mse`). Our models were trained on 60,000 images approximately, for 200,000 iterations with a batch size of 16. The U-Net architecture contains approximately 12.9 million parameters.

We experimented with three SDE processes: VP, subVP, and VE. For the latter two we tested three specific training objectives:

- Baseline: standard score matching objective.
- Likelihood Weighting (LW): minimizing the upper bound on the negative log-likelihood as derived by (8).
- LW + Importance Sampling (IS): LW combined with IS reduce variance of the loss estimator.

Furthermore, for the subVP SDE we analyssed how different guidance scales $\omega \in \{1.5, 3.5, 5, 7.5\}$ impacted performance of the CFG.

To ensure an adeguate metrics evaluation, we computed the NLL on 1,000 held out images and generate 10,000 samples to calculate FID and IS scores. It is important to note that in our implementation NLL specifically measures the prior matching term in Eq.14 in the latent space, since the reconstruction error is fixed by the pretrained VAE.

The actual implementation can be found (4)

10.2. Results and analysis

The Table 1 summarise the performance across different SDE and training configuartions.

LW and IS As shown in literature, exclusive LW training cause FID degradation. We suppose this is due to the fact that LW forces the model to allocate capacity to modeling high-frequency noise inherit in the VAE's encoder-decoder distribution, drifting away from meaningful signal, and increasing the risk of overfitting noise, as explained in App. C and visible in Fig. 6.

Unexpectedly, we observe that there is a negligible divergence of performance between baseline models and LW+IS ones, which contrast with (8). We attribute this difference to the nature of the latent space itself. Since VAE already force the latent distribution, $p(\mathbf{z})$ towards $\mathcal{N}(0, \mathbf{I})$ the diffusion task is simplified, where essentially the model learns a mapping from a noisy Gaussian to a less noisy gaussian. In this situation the weighting function $\lambda(t)$ seem to play a less

SDE	Guid.	Model	FID	IS	NLL
SubVP	1.5	Base	58.38	3.23	25 ± 14
SubVP	3.5	Base	62.16	3.54	-
SubVP	5	Base	67.34	3.78	-
SubVP	7.5	Base	77.86	4.09	-
SubVP	1.5	LW	67.16	3.11	29.16 ± 11
SubVP	3.5	LW	67.24	3.3	-
SubVP	5	LW	67.48	3.38	-
SubVP	7.5	LW	69.01	3.5	-
SubVP	1.5	LW+IS	**56.7**	2.86	26.78 ± 12
SubVP	3.5	LW+IS	59.16	3.29	-
SubVP	5	LW+IS	60.88	3.47	-
SubVP	7.5	LW+IS	65.92	3.76	-
SubVP Exp	1.5	LW+IS	60.44	2.96	26.78 ± 12
VP	1.5	Base	55.56	2.90	25.60 ± 13
VP	1.5	LW	69.59	3.00	30.44 ± 12
VP	1.5	LW+IS	**55.55**	2.90	25.60 ± 13
VE	1.5	Base	77.32	3.09	25.20 ± 14
Song SubVP	-	Base	8.71	-	3.82
Song SubVP	-	LW	12.99	-	3.82
Song SubVP	-	LW+IS	10.57	-	3.79
Song VP	-	Base	8.34	-	3.84
Song VP	-	LW	17.75	-	3.88
Song VP	-	LW+IS	11.15	-	3.80

Table 1. Comparison of Diffusion Models across SDE formulations and metrics. *NLL* is in *bits/dim*. Song's Models trained on the ImageNet 32x32 without Classifier-Free Guidance.

critical optimization stability role. Seemingly, the baseline model ignores fine-grained VAE artifacts by down-playing low t scores, while in the meantime LW+IS force attentions at all scales.

CFG and VAE misalignment Experimenting with different guidance scale tells us that, higher values improve IS, so the model ability to generate distinct and specific class features. However, this impair the FID scores, as it is possible to observe in Fig. 7.

We suppose this is due to a misalignment between latent distribution and the probabilistic decoder one, as shown in App. C. Particularly, high guidance scale values push the sampling trajectory towards high-density regions of the conditional distribution $p(\mathbf{z}|y)$, effectively pushing the latents into "extreme" region of the latent space, pushing $\mathbf{z}_0$ out of distribuiton, therefore causing artifacts. This cause lower perceptual quality (FID) but stronger semantic class signal (IS).

Comparison with benchmark our results differ from pixel-space baselines reported by Song et al. (8) (shown in Tab.1) and latent-space baselines by Rombach et al.(2). Specifically, the former operates directly in pixel space, where modeling high-frequency details is the primary bottleneck, while the latter train the Autoencoder from scratch alongside the diffusion prior, thus allowing the two components to adapt to each other. Our constraint of using a pretrained VAE introduces unique challenges, in particular related to the encoder noise artifacts and the decoder distribution mismatch, that influenced the optimal training strategy differently from others end-to-end approaches.

Some of our results, for instance LW only training degradation of FID agreed with literature, while others don't. Theoretically VE SDE provide greater expressivity due to unbounded variance, nevertheless our experiments yielded inferior metrics for this configuration (see Fig. 5). We attribute this underperformance to computational constraints, specifically the limited training duration of 70 epochs on 60,000 images, and the slower convergance of such SDE.

As discussed, we observed negligible performance divergence between the VP baseline and the VP model trained with LW + IS. This finding contrasts with results reported by Song et al. (8), where it is shown that while LW typically compromises sample quality in favor of likelihood, the baseline model outperform likelihood-weighted variants in terms of generation quality. This again has to do with the different space that our modeled, pixel and latent.

11. Limitations & Future work

Our implementation of LDM has revealed several constraints that derive directly from our computational limitations. A primary weakness arises from the necessity of maintaining small batch sizes due to GPU memory constraints in addition small dataset and a small number of iterations, only 200k approximately.
As we already explained, a second limitation is our reliance on a pretrained VAE, which, while reducing computational load, creates a structural bottleneck where the diffusion prior and the reconstruction module cannot adapt to each other, causing a possible Latent-Decoder Misalignment.

To address these implementation problems, possibile future work should incorporate the training from scratch of a Variational Autoencoder to eliminate the "matching prior" dominance and allow the model to better capture high-frequency details specific to the CelebA manifold (2).

While Diffusion Models represent the current state-of-the-art in terms of training stability and distributional accuracy, GANs maintain a significant advantage for sampling efficiency and image quality. Future research should aim to explore a possible hybrid architectures that combine the robustness of the diffusion process with the expressivity of GANs, aiming to mitigate image blur without sacrificing sample diversity (12).

12. Conclusion

This report detailed the development and evaluation of a LDM implemented with a continuous stochastic differential equation SDE framework. By moving the diffusion process from a high-dimensional pixel space to a compressed latent representation through a pretrained VAE, we maintained high perceptual quality while optimizing computational efficiency.
Ultimately, while the reliance on a fixed, pretrained VAE creates a structural limitations, this implementation shows that LDMs can be a robust and scalable solution for image synthesis under constrained resources.

References

[1] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021.

[2] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models, 2021.

[3] Max Welling Diederik P. Kingma. An introduction to variational autoencoders. *Foundations and Trends in Machine Learning: Vol 12, pp 307-392*, 2019.

[4] Giacomo Negri and Marco Zimmatore. Hands-on-generative-ai. https://github.com/zimmy11/Hands-on-Generative-AI, 2025.

[5] Pascal Vincent. A connection between score matching and denoising autoencoders. Technical report, Université de Montréal, 2010.

[6] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.

[7] Brian D.O. Anderson. Reverse-time diffusion equation models. *Stochastic Processes and their Applications, Elsevier*, 1982.

[8] Yang Song, Iain Murray, Conor Durkan, and Stefano Ermon. Maximum likelihood training of score-based diffusion models. *arXiv preprint arXiv:2101.09258*, 2021.

[9] Tim Salimans Jonathan Ho. Classifier-free diffusion guidance. In *NeurIPS*, 2022.

[10] Alex Nichol Prafulla Dhariwal. Diffusion models beat gans on image synthesis, 2021.

[11] Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. Deep learning face attributes in the wild. In *Proceedings of International Conference on Computer Vision (ICCV)*, December 2015.

[12] Jean-Yves Franceschi, Mike Gartrell, Ludovic Dos Santos, Thibaut Issenhuth, Emmanuel de Bézenac, Mickaël Chen, and Alain Rakotomamonjy. Unifying gans and score-based diffusion as generative particle models. 2023.

A. Images

In this section we will report some of the generated images and denoising processes that we have experimented with.

Figure 3. Denoising process with 1000 reverse steps for VP trained with LW + IS and 1.5 guidance. Reconstruction started from $\mathcal{N}(0, \mathbf{I})$ with batch prior mean of -0.005 and standard deviation of 0.999. The conditioning was of '*a young smiling male with straight hairs, bags under his eyes, and a 5 o'clock shadow*'.

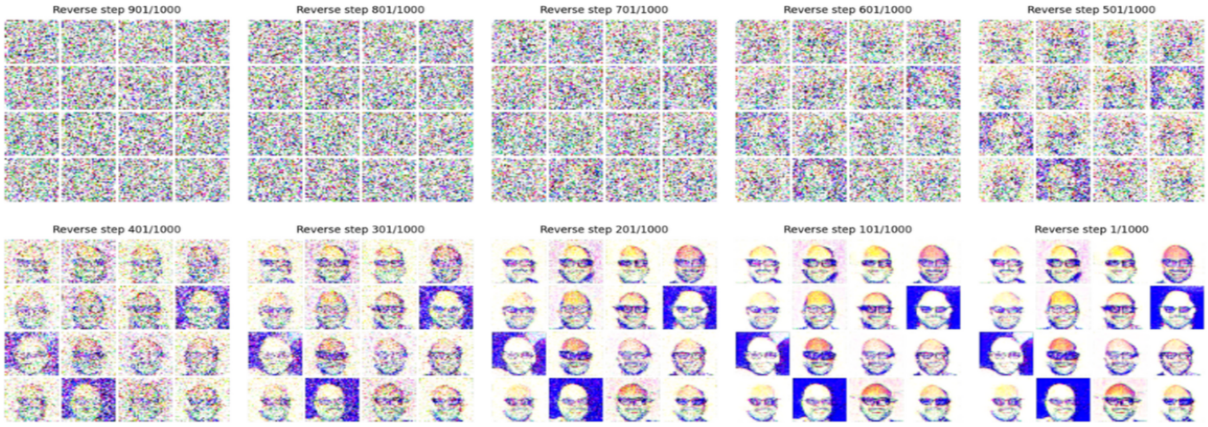

Figure 4. Denoising process with 1000 reverse steps for SubVP base trained with 1.5 guidance. Reconstruction started from $\mathcal{N}(0, \mathbf{I})$ with batch prior mean of 0.003 and standard deviation of 0.997. The conditioning used was '*a young male, bald, with a 5 o'clock shadow, bags under eyes, big nose, blurry, bushy eyebrows, eyeglasses, goatee, mouth slightly open, mustache, narrow eyes, oval face, pointy nose, receding hairline, sideburns, smiling*'.

The subsequent tests were generated using the following conditioning: she should be a young, attractive woman with arched eyebrows, brown straight hair, high cheekbones, and a pointy nose, wearing heavy makeup and lipstick and earrings, smiling with her mouth slightly open and with no beard; she should not have a 5 o'clock shadow, bags under her eyes, baldness or a receding hairline, bangs, big lips or a big nose, black/blond/gray hair, bushy eyebrows, a chubby face, a double chin, eyeglasses, goatee, mustache, sideburns, narrow eyes, an oval face, pale skin, rosy cheeks, wavy hair, or be wearing a hat, necklace, or necktie, and the image should not be blurry.

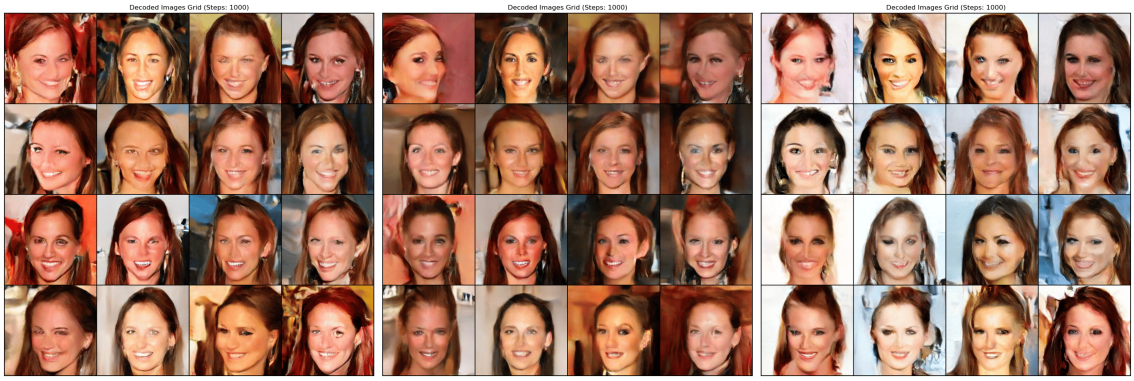

Figure 5. Denoising process with 1000 reverse steps for VP trained with LW + IS and 1.5 guidance (left), subVP trained with LW + IS and 1.5 guidance (center), and VE with 1.5 guidance (right). Reconstruction started from $\mathcal{N}(0, \mathbf{I})$ for VP and subVP, and $\mathcal{N}(0, \sigma^2_{\text{max}} = 50.0)$ for VE.

Figure 6. Denoising process with 1000 reverse steps for subVP base and 1.5 guidance (left), subVP trained with LW and 1.5 guidance (center), and subVP trained with LW + IS with 1.5 guidance (right). Reconstruction started from $\mathcal{N}(0, \mathbf{I})$.

Figure 7. Comparison of generated samples via subVP SDE across different Guidance Scales (g). In order, $g = 1.5, g = 3.5, g = 5, g = 7.5$, from left to right.

B. Mathematical Proofs

B.1. Derivation of Score Matching from Fisher Divergence (leading to Eq. (2))

Let $p(x)$ be the unknown data density on $\mathbb{R}^d$, and let $s_\theta(x)$ be our learned score intended to approximate the score $\nabla_x \log p(x)$. Consider the Fisher divergence, or score matching divergence, between p and the implicit model s_θ:

$$\mathcal{J}_F(\theta) := \frac{1}{2}\,\mathbb{E}_{p(x)}\left[\|s_\theta(x) - \nabla_x \log p(x)\|_2^2\right]. \tag{15}$$

Although $\nabla_x \log p(x)$ is intractable, we show that $\mathcal{J}_F(\theta)$ is equivalent, up to a θ independent constant, to a tractable objective depending only on s_θ and its divergence.

Step 1: expand the square and isolate θ-dependent terms Expanding Eq. (15) gives

$$\mathcal{J}_F(\theta) = \frac{1}{2}\,\mathbb{E}_p\big[\|s_\theta(x)\|_2^2\big] - \mathbb{E}_p\big[s_\theta(x)^\top \nabla_x \log p(x)\big] + \frac{1}{2}\,\mathbb{E}_p\big[\|\nabla_x \log p(x)\|_2^2\big]. \tag{16}$$

The last term does not depend on θ; hence it can be absorbed into a constant. The only problematic term is $\mathbb{E}_p[s_\theta^\top \nabla \log p]$.

Step 2: rewrite the cross term without $\nabla \log p$ Using $\nabla_x \log p(x) = \frac{\nabla_x p(x)}{p(x)}$, we obtain

$$\mathbb{E}_p\big[s_\theta(x)^\top \nabla_x \log p(x)\big] = \int_{\mathbb{R}^d} p(x)\, s_\theta(x)^\top \frac{\nabla_x p(x)}{p(x)}\, dx = \int_{\mathbb{R}^d} s_\theta(x)^\top \nabla_x p(x)\, dx. \tag{17}$$

Now apply integration by parts in $\mathbb{R}^d$, and assuming standard regularity, or decay conditions, such that boundary terms goes to zero (vanish), which means $p(x)\, s_\theta(x) \to 0$ as $\|x\| \to \infty$. Then

$$\begin{aligned}\int_{\mathbb{R}^d} s_\theta(x)^\top \nabla_x p(x)\, dx &= \sum_{i=1}^d \int_{\mathbb{R}^d} s_{\theta,i}(x)\, \partial_{x_i} p(x)\, dx = -\sum_{i=1}^d \int_{\mathbb{R}^d} p(x)\, \partial_{x_i} s_{\theta,i}(x)\, dx \\ &= -\int_{\mathbb{R}^d} p(x)\, \nabla_x \cdot s_\theta(x)\, dx = -\mathbb{E}_p[\nabla_x \cdot s_\theta(x)]\,. \end{aligned} \tag{18}$$

Therefore,

$$\mathbb{E}_p\big[s_\theta(x)^\top \nabla_x \log p(x)\big] = -\mathbb{E}_p[\nabla_x \cdot s_\theta(x)]\,. \tag{19}$$

Step 3: obtain the tractable score matching objective (Eq. (2)) Substitute Eq. (19) into Eq. (16):

$$\mathcal{J}_F(\theta) = \frac{1}{2}\,\mathbb{E}_p\big[\|s_\theta(x)\|_2^2\big] + \mathbb{E}_p[\nabla_x \cdot s_\theta(x)] + \underbrace{\frac{1}{2}\,\mathbb{E}_p\big[\|\nabla_x \log p(x)\|_2^2\big]}_{\text{const w.r.t. } \theta}. \tag{20}$$

Dropping the θ-independent constant yields the tractable score matching loss

$$J(\theta) = \mathbb{E}_{p(x)}\left[\frac{1}{2}\|s_\theta(x)\|_2^2 + \nabla_x \cdot s_\theta(x)\right], \tag{21}$$

which matches Eq. (2). Here $\nabla_x \cdot s_\theta(x) = \mathrm{Tr}\big(\nabla_x s_\theta(x)\big)$ is the divergence (trace of the Jacobian).

Remark Eq. (21) is equivalent to minimizing the Fisher divergence $\mathcal{J}_F(\theta)$ because the two objectives differ only by an additive constant independent of θ. Hence, the minimizer of $J(\theta)$ satisfies $s_{\theta^\star}(x) = \nabla_x \log p(x)$ almost everywhere under p.

B.2. Derivation of Denoising Score Matching (DSM)

In the previous section, starting from the Fisher divergence $\frac{1}{2}\,\mathbb{E}_{p(x)}\big[\|s_\theta(x) - \nabla_x \log p(x)\|_2^2\big]$, we derived the tractable score matching objective $\mathbb{E}_{p(x)}\big[\frac{1}{2}\|s_\theta(x)\|_2^2 + \nabla_x \cdot s_\theta(x)\big]$, which avoids the intractable term $\nabla_x \log p(x)$. Denoising Score Matching (DSM) extends this idea by replacing the unknown density $p(x)$ with a perturbed, smoothed, density $p_\sigma(\tilde{x})$ whose score can be learned using only a tractable conditional score of the perturbation kernel.

Perturbation and smoothed density Let $p_{\text{data}}(x)$ be the data distribution and let $q_\sigma(\tilde{x} \mid x)$ be a known perturbation kernel, Gaussian in our case. Define the σ-smoothed density

$$p_\sigma(\tilde{x}) \;:=\; \int p_{\text{data}}(x)\, q_\sigma(\tilde{x} \mid x)\, dx. \tag{22}$$

Our aim is to learn $s_\theta(\tilde{x}) \approx \nabla_{\tilde{x}} \log p_\sigma(\tilde{x})$, but $\nabla_{\tilde{x}} \log p_\sigma(\tilde{x})$ is still intractable because p_σ involves an integral over p_{data}.

Step 1: score matching on the perturbed density p_σ Applying the Fisher-divergence formulation to p_σ yields the intractable objective

$$\mathcal{J}_\sigma(\theta) := \frac{1}{2}\, \mathbb{E}_{\tilde{x}\sim p_\sigma}\left[\|s_\theta(\tilde{x}) - \nabla_{\tilde{x}} \log p_\sigma(\tilde{x})\|_2^2\right]. \tag{23}$$

As before, the difficulty is the unknown score $\nabla_{\tilde{x}} \log p_\sigma(\tilde{x})$. DSM removes this term by expressing $\nabla_{\tilde{x}} \log p_\sigma(\tilde{x})$ through the conditional score of the known kernel $q_\sigma(\tilde{x} \mid x)$.

Step 2: a tractable expression for $\nabla_{\tilde{x}} \log p_\sigma(\tilde{x})$ We compute the score of the smoothed density p_σ. Differentiating Eq. (22) under the integral sign,

$$\begin{aligned}\nabla_{\tilde{x}} p_\sigma(\tilde{x}) &= \nabla_{\tilde{x}} \int p_{\text{data}}(x)\, q_\sigma(\tilde{x} \mid x)\, dx = \int p_{\text{data}}(x)\, \nabla_{\tilde{x}} q_\sigma(\tilde{x} \mid x)\, dx \\ &= \int p_{\text{data}}(x)\, q_\sigma(\tilde{x} \mid x)\, \nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x)\, dx.\end{aligned} \tag{24}$$

Dividing by $p_\sigma(\tilde{x})$ gives

$$\begin{aligned}\nabla_{\tilde{x}} \log p_\sigma(\tilde{x}) &= \frac{1}{p_\sigma(\tilde{x})} \int p_{\text{data}}(x)\, q_\sigma(\tilde{x} \mid x)\, \nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x)\, dx \\ &= \int \frac{p_{\text{data}}(x)\, q_\sigma(\tilde{x} \mid x)}{p_\sigma(\tilde{x})}\, \nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x)\, dx = \mathbb{E}[\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x) \mid \tilde{x}]\,,\end{aligned} \tag{25}$$

where we used Bayes' rule to identify the posterior

$$p(x \mid \tilde{x}) = \frac{p_{\text{data}}(x)\, q_\sigma(\tilde{x} \mid x)}{p_\sigma(\tilde{x})}. \tag{26}$$

Equation (25) is the key identity: the (intractable) score of p_σ equals a posterior expectation of the *tractable* conditional score of q_σ.

Step 3: from score matching on p_σ to DSM Starting from Eq. (23), expand the square:

$$\begin{aligned}\mathcal{J}_\sigma(\theta) &= \mathbb{E}_{\tilde{x}\sim p_\sigma}\left[\frac{1}{2}\|s_\theta(\tilde{x})\|_2^2 - s_\theta(\tilde{x})^\top \nabla_{\tilde{x}} \log p_\sigma(\tilde{x}) + \frac{1}{2}\|\nabla_{\tilde{x}} \log p_\sigma(\tilde{x})\|_2^2\right] \\ &= \mathbb{E}_{\tilde{x}\sim p_\sigma}\left[\frac{1}{2}\|s_\theta(\tilde{x})\|_2^2 - s_\theta(\tilde{x})^\top \nabla_{\tilde{x}} \log p_\sigma(\tilde{x})\right] + \text{const},\end{aligned} \tag{27}$$

where the constant term does not depend on θ. Now replace $\nabla_{\tilde{x}} \log p_\sigma(\tilde{x})$ using Eq. (25):

$$\begin{aligned}\mathbb{E}_{\tilde{x}\sim p_\sigma}\left[s_\theta(\tilde{x})^\top \nabla_{\tilde{x}} \log p_\sigma(\tilde{x})\right] &= \mathbb{E}_{\tilde{x}\sim p_\sigma}\left[s_\theta(\tilde{x})^\top \mathbb{E}[\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x) \mid \tilde{x}]\right] \\ &= \mathbb{E}_{x\sim p_{\text{data}}}\, \mathbb{E}_{\tilde{x}\sim q_\sigma(\cdot|x)}\left[s_\theta(\tilde{x})^\top \nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x)\right].\end{aligned} \tag{28}$$

Substituting Eq. (28) into Eq. (27) yields an equivalent objective (up to a θ-independent constant):

$$\mathcal{L}_{\text{DSM}}(\theta) \;:=\; \mathbb{E}_{x\sim p_{\text{data}}}\, \mathbb{E}_{\tilde{x}\sim q_\sigma(\cdot|x)}\left[\frac{1}{2}\|s_\theta(\tilde{x})\|_2^2 - s_\theta(\tilde{x})^\top \nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x)\right] + \text{const}. \tag{29}$$

Finally, completing the square inside the expectation shows that Eq. (29) is equivalent to the standard DSM form:

$$\mathcal{L}_{\text{DSM}}(\theta) = \mathbb{E}_{x\sim p_{\text{data}}}\, \mathbb{E}_{\tilde{x}\sim q_\sigma(\cdot|x)}\left[\frac{1}{2}\,\|s_\theta(\tilde{x}) - \nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x)\|_2^2\right] + \text{const}. \tag{30}$$

Therefore, minimizing $\mathcal{L}_{\mathrm{DSM}}(\theta)$ is equivalent, up to an additive constant, to minimizing the Fisher divergence between $s_\theta(\tilde{x})$ and the true score $\nabla_{\tilde{x}} \log p_\sigma(\tilde{x})$. In particular, at the population optimum,

$$s_{\theta^\star}(\tilde{x}) = \nabla_{\tilde{x}} \log p_\sigma(\tilde{x}) \quad \text{a.e. under } p_\sigma. \tag{31}$$

Gaussian perturbation (closed-form target)o If $q_\sigma(\tilde{x} \mid x) = \mathcal{N}(\tilde{x}; x, \sigma^2 I)$, then

$$\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x) = -\frac{\tilde{x} - x}{\sigma^2}. \tag{32}$$

Using the reparameterization $\tilde{x} = x + \sigma\varepsilon$, $\varepsilon \sim \mathcal{N}(0, I)$, we obtain

$$\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x) = -\frac{\varepsilon}{\sigma}, \qquad \Rightarrow \qquad s_\theta(\tilde{x}, \sigma) \approx -\frac{\varepsilon_\theta(\tilde{x}, \sigma)}{\sigma}, \tag{33}$$

which justifies the common noise-prediction parameterization used in diffusion/score-based generative models.

B.3. Derivation of perturbation kernels for VP, VE, subVP

We consider a forward Itô SDE

$$dx_t = f(x_t, t)\, dt + g(t)\, dw_t, \tag{34}$$

where w_t is standard Brownian motion. For VE/VP/subVP, the drift is affine in x_t, hence the transition $p_{0t}(x_t \mid x_0)$ is Gaussian and can be written in the unified form

$$p_{0t}(x_t \mid x_0) = \mathcal{N}\big(x_t;\, \mu(t)x_0,\, \sigma^2(t)I\big). \tag{35}$$

below we derive $\mu(t)$ and $\sigma^2(t)$ for each SDE.

Variance Exploding (VE) SDE the VE SDE is

$$dx_t = \sqrt{\frac{d\,\sigma^2(t)}{dt}}\, dw_t, \tag{36}$$

where $\sigma(t)$ is an increasing noise scale. Integrating Eq. (36) from 0 to t yields

$$x_t = x_0 + \int_0^t \sqrt{\frac{d\,\sigma^2(s)}{ds}}\, dw_s. \tag{37}$$

The stochastic integral is Gaussian with mean 0 and covariance

$$\mathrm{Var}(x_t \mid x_0) = \left(\int_0^t \frac{d\,\sigma^2(s)}{ds}\, ds\right) I = \big(\sigma^2(t) - \sigma^2(0)\big) I. \tag{38}$$

If $\sigma(0) = 0$, which is a common convention (or approximately zero), then

$$p_{0t}^{\mathrm{VE}}(x_t \mid x_0) = \mathcal{N}\big(x_t;\, x_0,\, \sigma^2(t)I\big), \qquad \mu(t) = 1,\ \sigma^2(t) = \sigma^2(t). \tag{39}$$

Variance Preserving (VP) SDE the VP SDE is linear:

$$dx_t = -\frac{1}{2}\beta(t)\, x_t\, dt + \sqrt{\beta(t)}\, dw_t, \tag{40}$$

with a noise rate $\beta(t) > 0$. Define the integrated rate

$$B(t) := \int_0^t \beta(s)\, ds, \qquad \mu(t) := \exp(-2B(t))\,. \tag{41}$$

Use the integrating factor $\mu(t)^{-1} = \exp(+2B(t))$:

$$d\big(\mu(t)^{-1}x_t\big) = \mu(t)^{-1}\sqrt{\beta(t)}\,dw_t, \tag{42}$$

so

$$x_t = \mu(t)x_0 + \mu(t)\int_0^t \mu(s)^{-1}\sqrt{\beta(s)}\,dw_s. \tag{43}$$

The mean is $\mathbb{E}[x_t \mid x_0] = \mu(t)x_0$. The conditional covariance is

$$\mathrm{Var}(x_t \mid x_0) = \mu(t)^2\int_0^t \mu(s)^{-2}\beta(s)\,ds\; I = \Big(1 - e^{-B(t)}\Big)\,I, \tag{44}$$

where the last equality follows by differentiating $e^{-B(t)}$ and checking the ODE. Therefore,

$$p_{0t}^{\mathrm{VP}}(x_t \mid x_0) = \mathcal{N}\Big(x_t;\ \mu(t)x_0,\ \big(1 - e^{-B(t)}\big)I\Big), \qquad \sigma^2(t) = 1 - e^{-B(t)}. \tag{45}$$

subVP SDE (discounted variance) The subVP SDE uses the same drift as VP but a reduced diffusion to bound the variance:

$$dx_t = -\frac{1}{2}\beta(t)\,x_t\,dt + \sqrt{\beta(t)\Big(1 - e^{-2B(t)}\Big)}\,dw_t, \tag{46}$$

with $B(t)$ as in Eq. (41). The mean is unchanged because the drift is unchanged: $\mathbb{E}[x_t \mid x_0] = \mu(t)x_0$, with $\mu(t) = \exp(-\frac{1}{2}B(t))$.

To get the variance, it is convenient to use the covariance ODE for linear SDEs. Let $V(t) := \mathrm{Var}(x_t \mid x_0)$ (per-coordinate; isotropic). The variance evolution is governed by:

$$\frac{dV}{dt} = -\beta(t)\,V(t) + \beta(t)\Big(1 - e^{-2B(t)}\Big), \qquad V(0) = 0. \tag{47}$$

Claim: the solution is

$$V(t) = \Big(1 - e^{-B(t)}\Big)^2. \tag{48}$$

Verification: differentiate Eq. (48) using $\frac{d}{dt}e^{-B(t)} = -\beta(t)e^{-B(t)}$:

$$\frac{dV}{dt} = 2\Big(1 - e^{-B(t)}\Big)\cdot\Big(-\frac{d}{dt}e^{-B(t)}\Big) \tag{49}$$

$$= 2\Big(1 - e^{-B(t)}\Big)\cdot\beta(t)e^{-B(t)} \tag{50}$$

$$= \beta(t)\Big(2e^{-B(t)} - 2e^{-2B(t)}\Big). \tag{51}$$

On the other hand, substituting the proposed $V(t)$ into the RHS of the ODE (47):

$$-\beta(t)V(t) + \beta(t)\big(1 - e^{-2B(t)}\big) = -\beta(t)\Big(1 - e^{-B(t)}\Big)^2 + \beta(t)\big(1 - e^{-2B(t)}\big) \tag{52}$$

$$= -\beta(t)\big(1 - 2e^{-B(t)} + e^{-2B(t)}\big) + \beta(t)\big(1 - e^{-2B(t)}\big) \tag{53}$$

$$= -\beta(t) + 2\beta(t)e^{-B(t)} - \beta(t)e^{-2B(t)} + \beta(t) - \beta(t)e^{-2B(t)} \tag{54}$$

$$= \beta(t)\Big(2e^{-B(t)} - 2e^{-2B(t)}\Big), \tag{55}$$

which matches. Thus $V(t)$ in Eq. (48) is correct, and the transition kernel is:

$$p_{0t}^{\mathrm{subVP}}(x_t \mid x_0) = \mathcal{N}\Big(x_t;\ \mu(t)x_0,\ \big(1 - e^{-B(t)}\big)^2 I\Big), \qquad \sigma^2(t) = \big(1 - e^{-B(t)}\big)^2. \tag{56}$$

B.4. Likelihood Weighting bound proof (continuous-time)

In this appendix it is proved the key identity behind likelihood weighting, where the KL divergence between two evolving densities decreases at a rate given by a (relative) Fisher information term. Integrating this rate results in a bound/identity that motivates choosing $\lambda(t) = g(t)^2$ in the continuous score-matching objective.

Setup Consider the forward SDE

$$dx_t = f(x_t, t)\,dt + g(t)\,dw_t, \tag{57}$$

and let $p_t(x)$ and $q_t(x)$ be the marginal densities at time t when $x_0 \sim p_0$ and $x_0 \sim q_0$, respectively, under the same forward SDE. Assume sufficient smoothness and decay so that integration by parts is valid and boundary terms vanish.

Both p_t and q_t satisfy the Fokker–Planck equation

$$\partial_t r_t = -\nabla \cdot (f\, r_t) + \frac{1}{2} g(t)^2 \Delta r_t, \qquad r_t \in \{p_t, q_t\}. \tag{58}$$

Claim

$$\frac{d}{dt} D_{\mathrm{KL}}(p_t \| q_t) = -\frac{1}{2} g(t)^2\, \mathbb{E}_{x\sim p_t}\left[\|\nabla_x \log p_t(x) - \nabla_x \log q_t(x)\|_2^2 \right]. \tag{59}$$

Proof start from

$$D_{\mathrm{KL}}(p_t \| q_t) = \int p_t(x)\, \log \frac{p_t(x)}{q_t(x)}\, dx. \tag{60}$$

differentiate w.r.t. t:

$$\begin{aligned} \frac{d}{dt} D_{\mathrm{KL}}(p_t \| q_t) &= \int (\partial_t p_t)\, \log \frac{p_t}{q_t}\, dx + \int p_t \left(\frac{\partial_t p_t}{p_t} - \frac{\partial_t q_t}{q_t} \right) dx \\ &= \int (\partial_t p_t)\, \log \frac{p_t}{q_t}\, dx + \int \partial_t p_t\, dx - \int p_t\, \partial_t \log q_t\, dx. \end{aligned} \tag{61}$$

Since $\int \partial_t p_t\, dx = \partial_t \int p_t\, dx = \partial_t(1) = 0$, Eq. (61) becomes

$$\frac{d}{dt} D_{\mathrm{KL}}(p_t \| q_t) = \int (\partial_t p_t)\, \log \frac{p_t}{q_t}\, dx - \int p_t\, \partial_t \log q_t\, dx. \tag{62}$$

Now substitute $\partial_t p_t$ and $\partial_t q_t$ from the Fokker–Planck equation (58). A "standard" but slightly lengthy integration-by-parts manipulation shows that the drift terms cancel out, and the diffusion terms results in exactly the following

$$\frac{d}{dt} D_{\mathrm{KL}}(p_t \| q_t) = -\frac{1}{2} g(t)^2 \int p_t(x)\, \|\nabla \log p_t(x) - \nabla \log q_t(x)\|_2^2\, dx, \tag{63}$$

which is Eq. (59). □

Integrated form integrating Eq. (59) from 0 to T gives

$$D_{\mathrm{KL}}(p_0 \| q_0) = D_{\mathrm{KL}}(p_T \| q_T) + \frac{1}{2} \int_0^T g(t)^2\, \mathbb{E}_{p_t}\left[\|\nabla \log p_t - \nabla \log q_t\|_2^2 \right] dt. \tag{64}$$

Connection to likelihood weighting in score-based diffusion, $p_0 = p_{\mathrm{data}}$. Let $q_0 = p_\theta$ be the model distribution at time 0, and choose the terminal condition $q_T = \pi$ (the prior). Then Eq. (64) becomes

$$D_{\mathrm{KL}}(p_{\mathrm{data}} \| p_\theta) = D_{\mathrm{KL}}(p_T \| \pi) + \frac{1}{2} \int_0^T g(t)^2\, \mathbb{E}_{p_t}\left[\|\nabla \log p_t - \nabla \log q_t\|_2^2 \right] dt. \tag{65}$$

If the score model $s_\theta(x, t)$ exactly equals $\nabla \log q_t(x)$ for all t, then minimizing the functional

$$J_{\mathrm{SM}}(\theta; g^2) := \frac{1}{2} \int_0^T \mathbb{E}_{p_t}\left[g(t)^2\, \|\nabla \log p_t(x) - s_\theta(x, t)\|_2^2 \right] dt \tag{66}$$

is equivalent, up to the θ-independent constant $D_{\mathrm{KL}}(p_T \| \pi)$, to minimizing $D_{\mathrm{KL}}(p_{\mathrm{data}} \| p_\theta)$, or maximum likelihood training in continuous time case. In practice s_θ is approximate, so Eq. (66) is used as a surrogate objective; its specific choice $\lambda(t) = g(t)^2$ is what we typically call Likelihood Weighting.

B.5. Probability Flow ODE derivation

In this appendix it is derived the probability flow ODE, whose deterministic dynamics marginals match those of the stochastic diffusion.

Forward SDE and its density evolution consider the forward SDE

$$dx_t = f(x_t, t)\, dt + g(t)\, dw_t, \tag{67}$$

with marginal density $p_t(x)$. The density evolves according to the Fokker–Planck equation

$$\partial_t p_t(x) = -\nabla \cdot \big(f(x,t)\, p_t(x)\big) + \frac{1}{2} g(t)^2 \Delta p_t(x). \tag{68}$$

Rewrite the diffusion term as a divergence use the identity $\Delta p = \nabla \cdot (\nabla p) = \nabla \cdot (p \nabla \log p)$ (valid where $p > 0$):

$$\Delta p_t = \nabla \cdot \big(p_t \nabla \log p_t\big). \tag{69}$$

substitute Eq. (69) into Eq. (68):

$$\begin{aligned} \partial_t p_t &= -\nabla \cdot (f\, p_t) + \frac{1}{2} g(t)^2 \nabla \cdot \big(p_t \nabla \log p_t\big) \\ &= -\nabla \cdot \left(\left[f(x,t) - \frac{1}{2} g(t)^2 \nabla \log p_t(x) \right] p_t(x) \right). \end{aligned} \tag{70}$$

Identify a continuity equation for a deterministic ODE

$$\frac{dx_t}{dt} = v(x_t, t), \tag{71}$$

the density satisfies the continuity (Liouville) equation

$$\partial_t p_t = -\nabla \cdot (v\, p_t). \tag{72}$$

comparing Eq. (72) with Eq. (70), we read off the required velocity field:

$$v(x,t) = f(x,t) - \frac{1}{2} g(t)^2 \nabla_x \log p_t(x). \tag{73}$$

Probability flow ODE therefore, the *probability flow ODE* associated with the SDE (67) is

$$\frac{dx_t}{dt} = f(x_t, t) - \frac{1}{2} g(t)^2 \nabla_{x_t} \log p_t(x_t). \tag{74}$$

By construction, the ODE (74) induces the *same marginal densities* $\{p_t\}_{t\in[0,T]}$ as the SDE (67); only the path-wise randomness is removed.

Plug-in score model in practice, we replace the unknown score $\nabla \log p_t(x)$ by the learned score network $s_\theta(x,t)$, yielding the implementable drift

$$\tilde{f}_\theta(x,t) := f(x,t) - \frac{1}{2} g(t)^2 s_\theta(x,t), \qquad \frac{dx_t}{dt} = \tilde{f}_\theta(x_t, t). \tag{75}$$

This is the ODE used for exact likelihood computation via change of variables once $\tilde{f}_\theta$ is fixed.

C. Latent Misalignment Risk

Our implementation relies on a *fixed* pretrained VAE to define the latent space and on a diffusion prior to model the latent distribution. This separation is computationally useful, but it introduces a structural risk: the diffusion model can learn a latent distribution that is internally consistent for denoising, yet partially incompatible with what the decoder can reliably map back to pixel space. In the main report we already observed this effect qualitatively when discussing guidance and artifacts, and we referred to it as a possible Latent-Decoder Misalignment.

C.1. Matching the two distirbutions

Let $q_\phi(z \mid x)$ be the VAE encoder posterior and let

$$q_\phi(z) = \int q_\phi(z \mid x)\, p_{\text{data}}(x)\, dx \tag{76}$$

be the aggregated posterior (the empirical latent distribution induced by the dataset through the encoder). The diffusion prior is trained to approximate (a noised version of) this latent distribution and then to sample from a learned model $p_\theta(z)$ via the reverse process.

The decoder, however, is trained to reconstruct images from encoder-produced latents, meaning $z \sim q_\phi(z \mid x)$ and therefore mostly from the high-density region of $q_\phi(z)$. As a consequence, good likelihood under $p_\theta(z)$ does not guarantee a stable decoding unless the sampling trajectory remains within the region where the decoder learned a well-behaved inverse mapping.

C.2. What is meant by misalignment

We call latent misalignment the situation in which

$$z \sim p_\theta(z) \quad \text{but} \quad z \notin \mathcal{S}_{\text{dec}}$$

where $\mathcal{S}_{\text{dec}}$ denotes the subset of latent space effectively covered during decoder training (high-density region of $q_\phi(z)$ and its typical perturbations). In practice this means the diffusion model can produce samples that are plausible under its own learned geometry, but are out-of-distribution (OOD) for the decoder.

C.3. Why it happens in latent diffusion pipelines

There are three common mechanisms.

(1) Aggregated posterior geometry is not "Gaussian" Even if each $q_\phi(z \mid x)$ has diagonal covariance, the mixture $q_\phi(z)$ can be anisotropic and globally correlated due to the variation of encoder means across data:

$$\text{Cov}_{q_\phi(z)}[z] = \mathbb{E}[\text{Cov}(z \mid x)] + \text{Cov}(\mathbb{E}[z \mid x])\,.$$

Empirically this often appears as a "thin" latent support: most variance is concentrated in a few directions, while many directions vary very little. The diffusion forward process adds isotropic noise, which easily pushes samples off this thin high-density region at small noise levels.

(2) Reverse-time discretization accumulates drift Reverse diffusion (SDE/ODE) is implemented with a finite-step solver, specifically Euler–Maruyama. Small score errors are injected at every step; even if each step is unbiased, the cumulative effect can move the trajectory away from typical regions of $q_\phi(z)$, especially near the end of sampling (small effective noise), where the density is sharply concentrated.

(3) Guidance amplifies off-manifold moves Under Classifier-Free Guidance,

$$s_{\text{cfg}}(x_t, t, c) = s_\theta(x_t, t, \varnothing) + \omega\big(s_\theta(x_t, t, c) - s_\theta(x_t, t, \varnothing)\big),$$

large ω extrapolates the score field. This can improve class specificity, but it also increases the probability of pushing the sample toward "extreme" conditional regions that were rarely (or never) visited by the decoder during training, which matches our observed IS/FID tradeoff.

C.4. Failure mode at decode time

Once the latent leaves $\mathcal{S}_{\text{dec}}$, the decoder behaves like a neural network extrapolator. Typical effects are not random: they are consistent with the decoder applying a learned inverse mapping outside its training support. Concretely, OOD latents can produce:

- texture or grid-like artifacts,
- washed-out contrast and incorrect colors,
- repeated patterns,
- local "melting" structure and loss of semantic coherence.

C.5. Why this matters

Latent misalignment is a systemic risk for modular LDM pipelines: it can degrade perceptual quality even when the diffusion loss and latent-space likelihood metrics look reasonable. It is also amplified by high guidance and by solvers with few sampling steps, since both increase trajectory extrapolation and numerical drift.

C.6. Practical mitigations (within our setup)

Without retraining the VAE end-to-end, the main levers are:

- **Moderate guidance**: keep ω in a regime where class signal improves without pushing latents into extreme regions.
- **More stable sampling**: increase the number of reverse steps and/or use a higher-order solver to reduce accumulated drift.
- **Latent regularization at sampling**: optional clipping/normalization of latents (or of predicted x_0) to stay closer to typical VAE latents, at the cost of some diversity.

A structural solution is to train the autoencoder jointly with the diffusion prior (or fine-tune it) so that the decoder remains calibrated to the latents produced by the learned prior, which we listed as a key future direction.